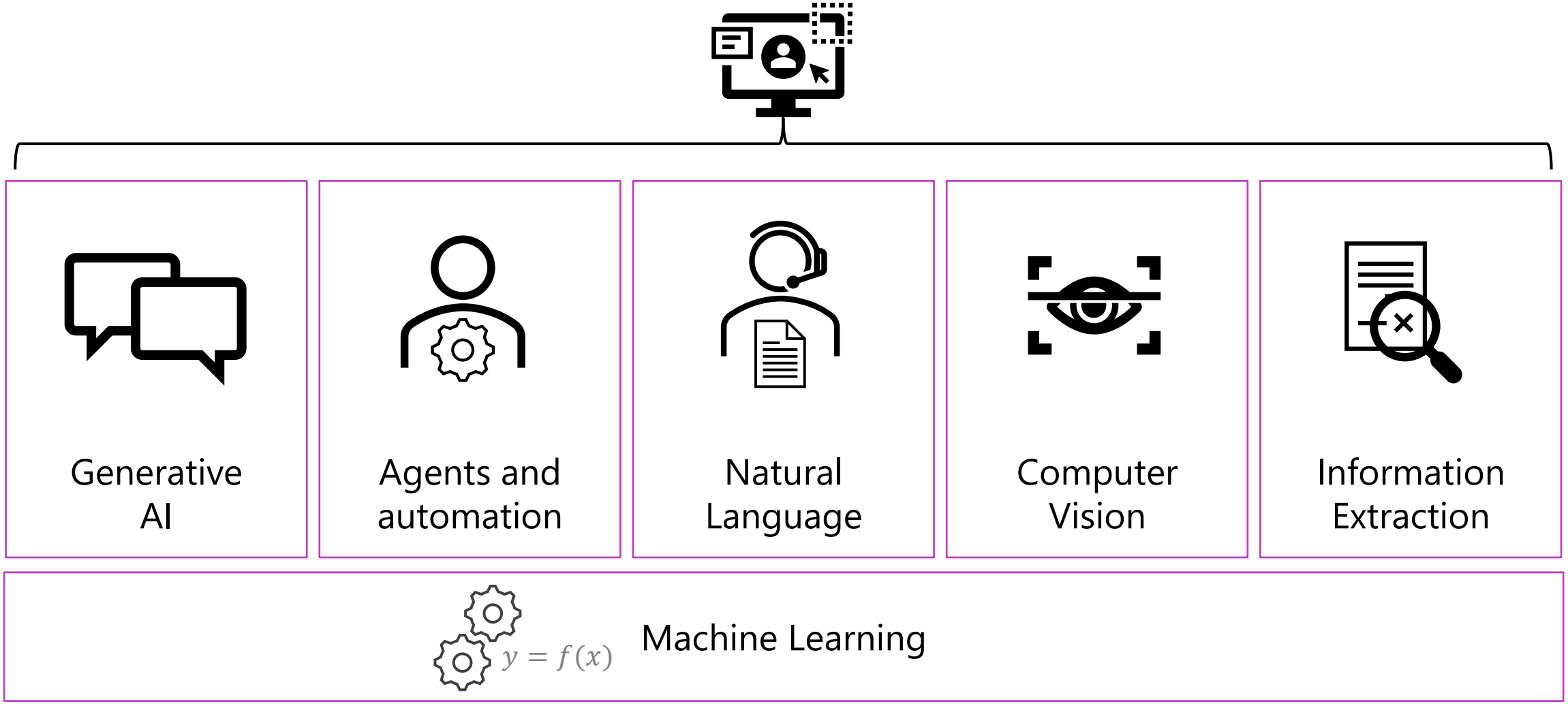
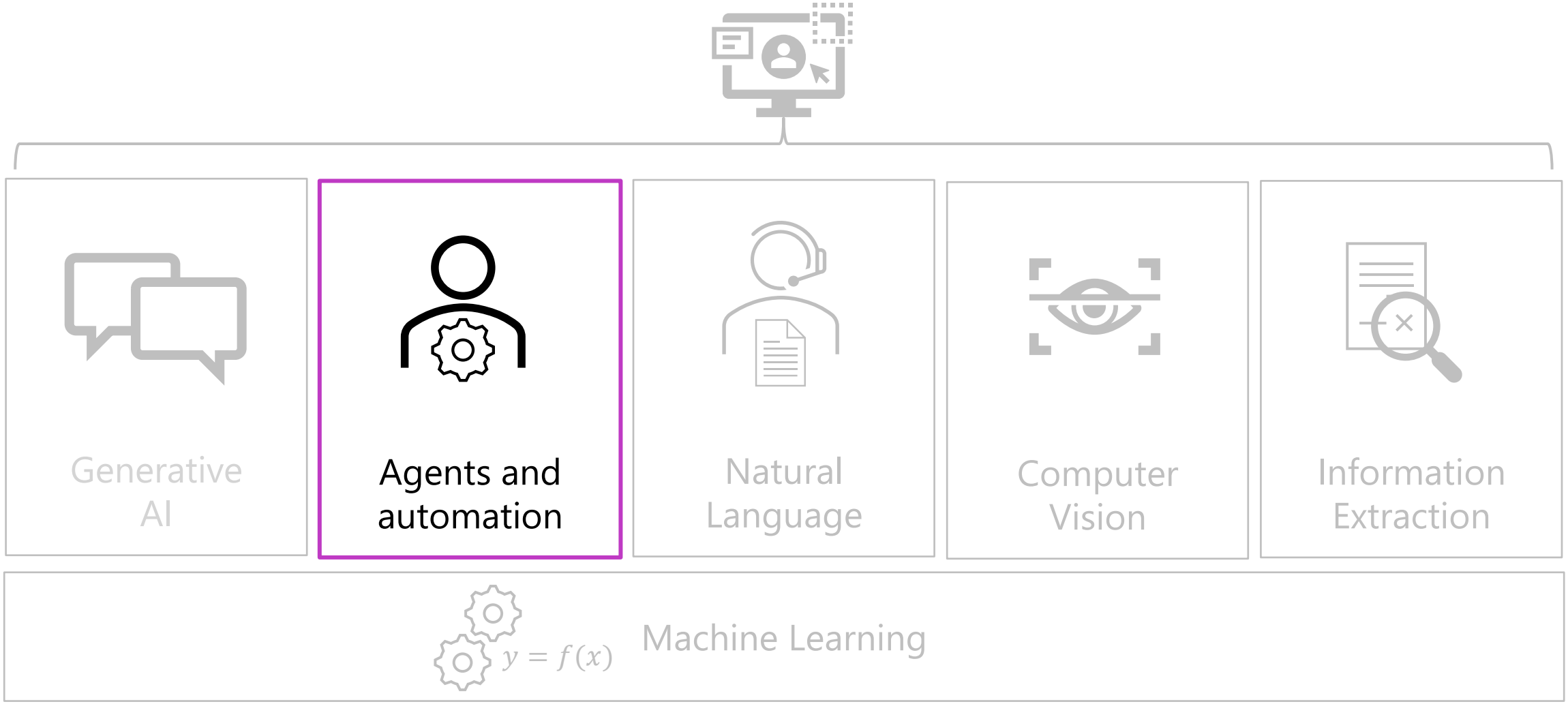
Microsoft

Azure AI Fundamentals & Building Your First Agent

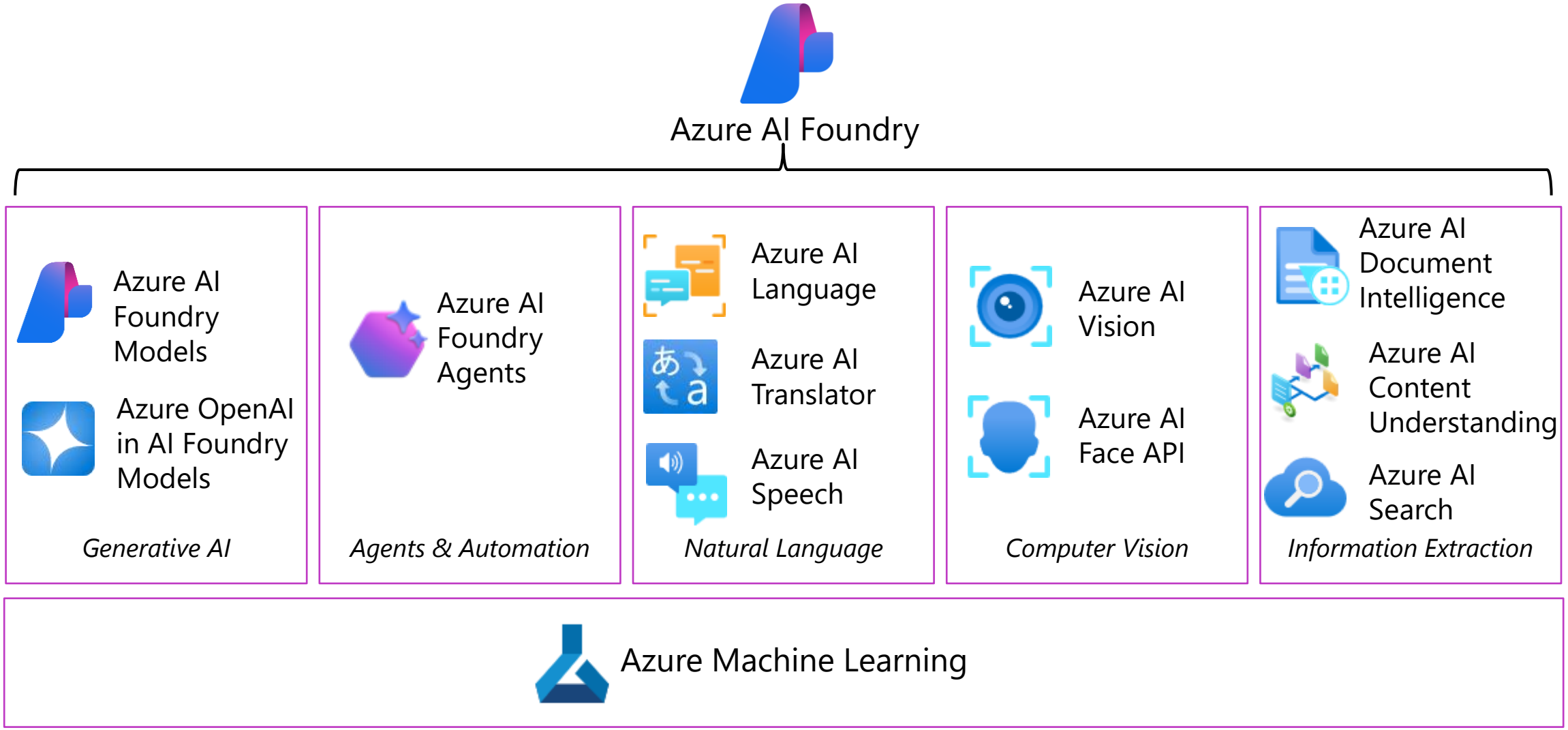
What is Artificial Intelligence?



Our focus today



AI services in Azure

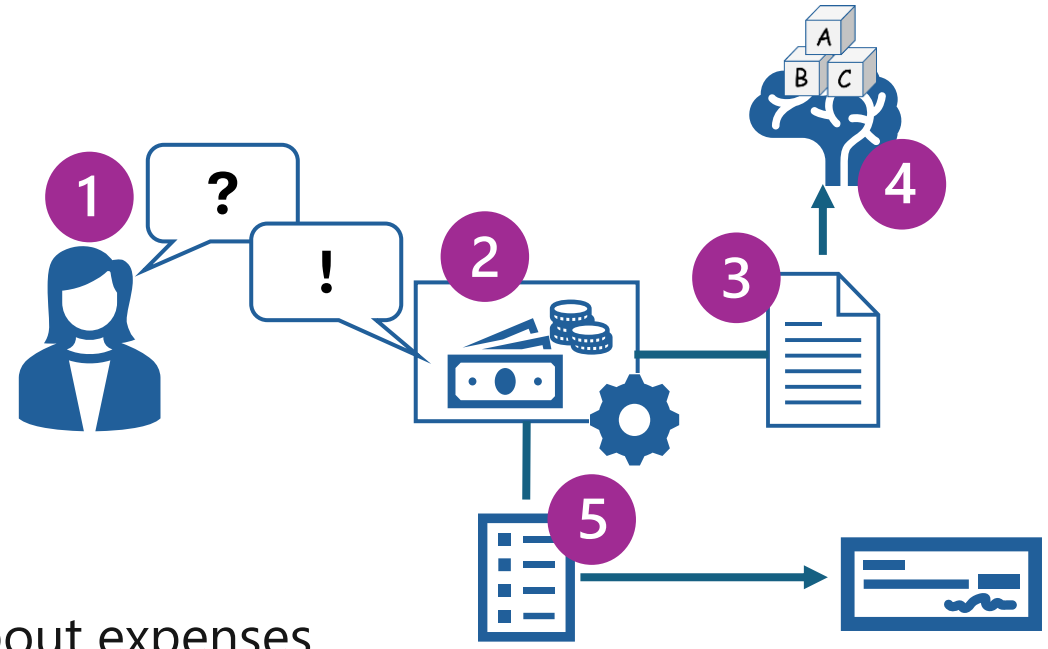


Get started with AI agent development on Azure

What are AI agents?

Smart software services that combine

- Generative AI models
- Contextual knowledge
- Tools to automate tasks



Example: An expenses agent

1. A user asks the expense agent a question about expenses
2. The expenses agent accepts the question as a prompt
3. The agent uses knowledge of expenses policy information to *ground* the prompt
4. The agent uses a *language model* to interpret the prompt and generate a response
5. Based on the response, the agent can use its available *tools* to submit a claim to be processed and generate a check payment

Options for agent development

Azure AI Foundry Agent Service

- Managed service in Azure for creating, managing, and using AI agents
- Based on OpenAI Assistants API - more model choices, data integration, and enterprise security

OpenAI Assistants API

- Standard API for OpenAI agent development
- Offers a subset of AI Agent Service features
- Restricted to OpenAI models

Semantic Kernel

- Lightweight, open-source development kit for building AI agents
- Supports OpenAI Assistant and Azure AI agents as well as native chat completion agents

AutoGen

- Open-source framework for rapid agent development
- Useful for research and ideation when experimenting with agents

Microsoft 365 Agents SDK

- Create self-hosted agents for various channels
- Not limited to Microsoft 365; can be delivered through channels like Slack or Messenger

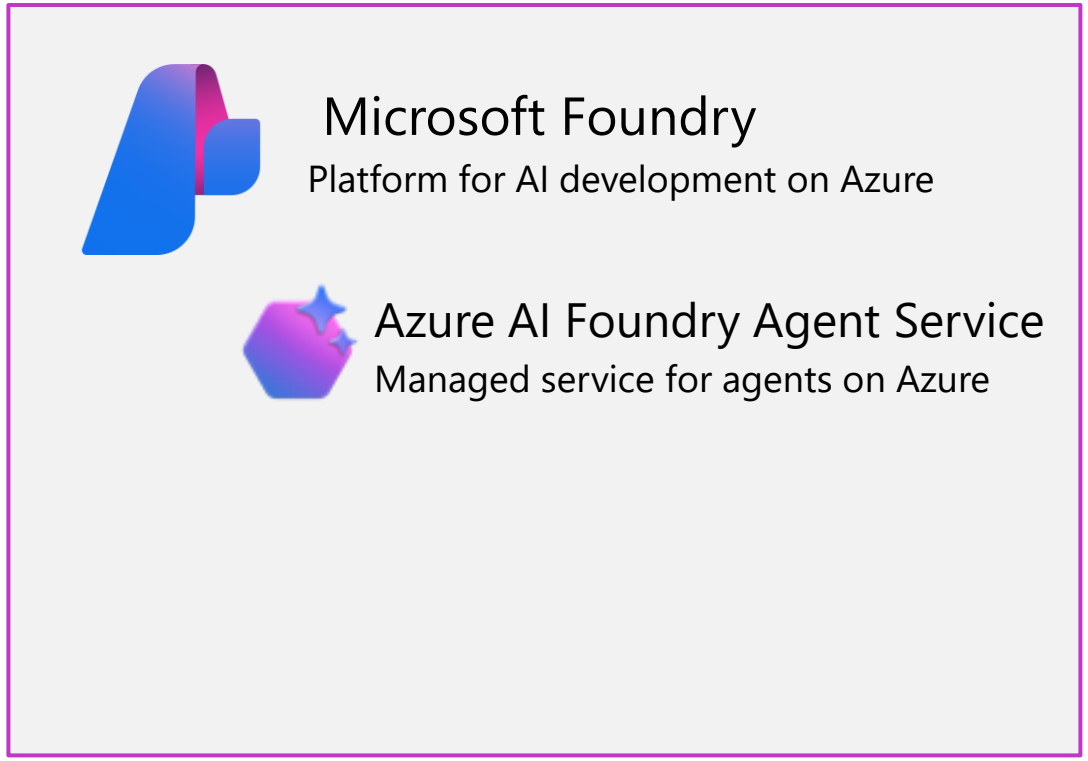
Copilot Studio

- Low-code development environment for building and deploying agents
- Integrates with Microsoft 365 ecosystem and common channels like Slack and Messenger

Agent Builder

- Declarative tool for business users to author basic agents in Microsoft Copilot Chat and other Copilot-based interfaces
- Enables users to create agents by describing functionality or using a visual interface

Our focus in this session



Azure AI Foundry provides a platform for agent development:

- Project-based resource management and model catalog
- Visual agent design and test in Azure AI Foundry portal
- Agent development with Azure AI Foundry Agent Service SDK
- Azure AI Agent Service integration with Semantic Kernel

Components of an agent

Model

- Generative AI language model
- Deployed from Azure AI model catalog
- Interprets prompts

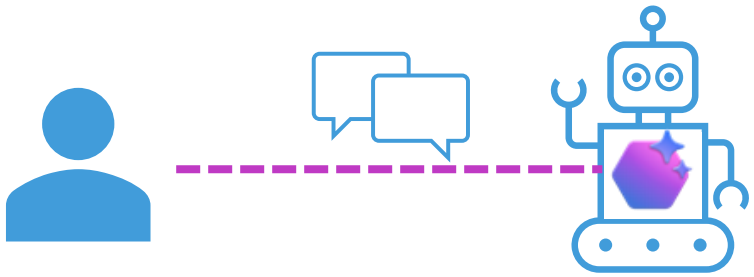
Tools

Knowledge tools

- Search information sources, like Bing and enterprise data stores

Action tools

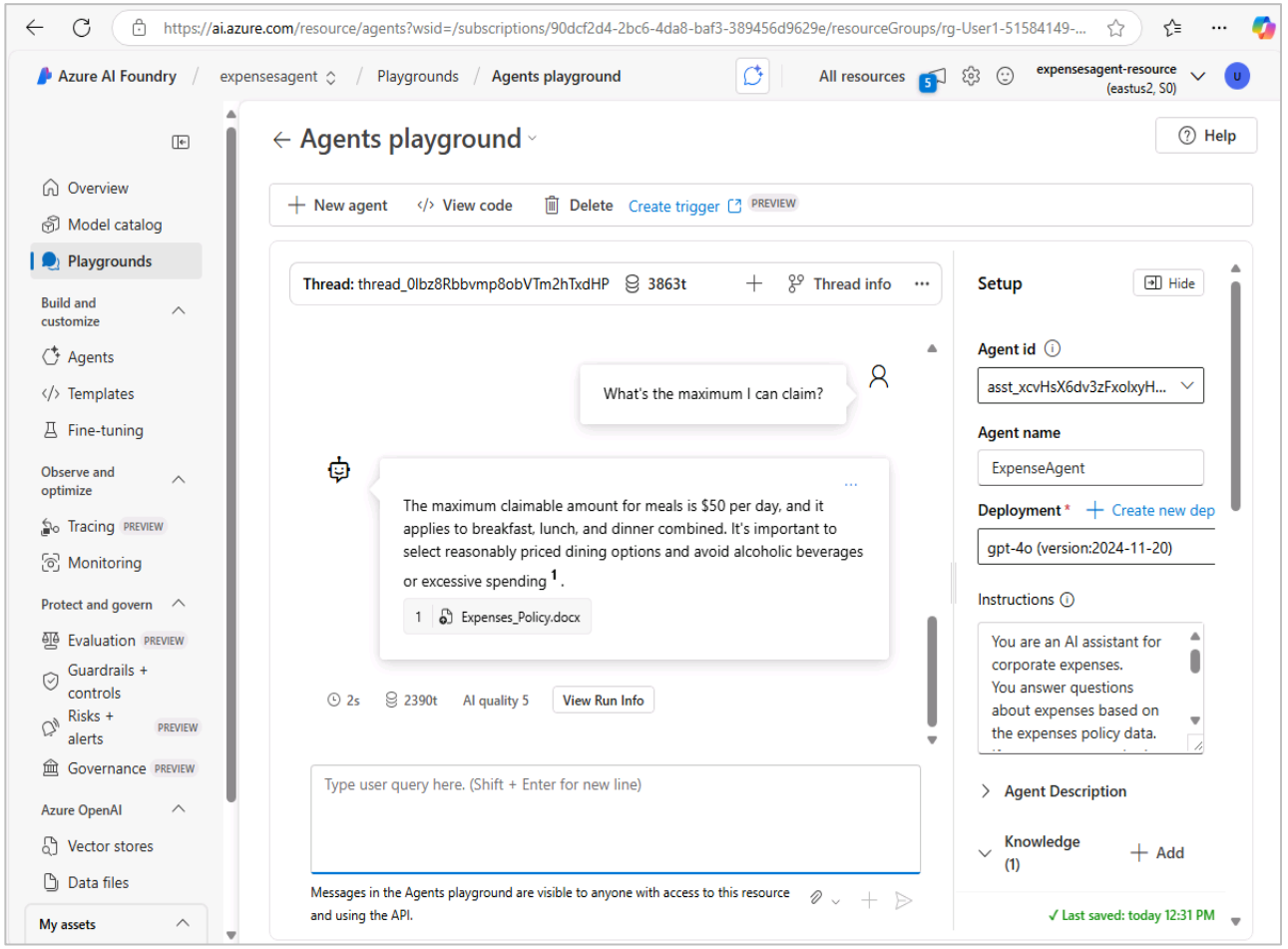
- Automate tasks, for example by generating and running code



Chat sessions with an agent take place on a *thread*

Creating an agent in Microsoft Foundry portal

1. Deploy a supported model
2. Create an agent and configure:
   - Name
   - Model deployment
   - Instructions
   - Tool connections
3. Test the agent in the playground
4. Connect to and consume your agent using code

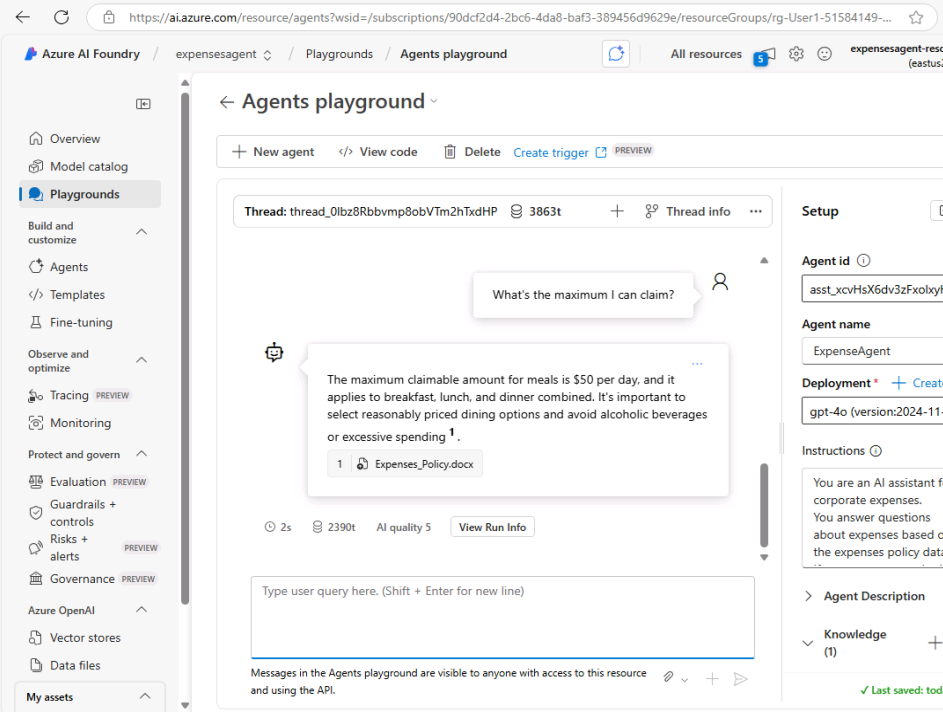


Exercise

Explore AI Agent development

In this exercise, you'll use the Azure AI Foundry portal to:

- Deploy a model
- Create an agent
- Configure a knowledge tool
- Test the agent in the playground



Knowledge check

1 **Which of the following best describes an AI agent?**

- ☐ A developer who specializes in building generative AI solutions
- ☑ A software service that uses AI to assist users with information and task automation
- ☐ A marketplace for off-the-shelf AI software components

2 **Which AI agent development service offers a choice of generative AI models from multiple vendors in the Azure AI Foundry model catalog?**

- ☑ Azure AI Foundry Agent Service
- ☐ OpenAI Assistants API
- ☐ Microsoft 365 Copilot Chat

3 **What element of an Foundry Agent Service agent enables it to ground prompts with contextual data?**

- ☐ Model
- ☐ Code interpreter tool
- ☑ Knowledge

Develop an AI agent with Azure AI Foundry Agent Service

Why are AI agents useful?

Automation

Handle repetitive tasks.
Free up human workers.
Increase productivity and efficiency.

Decision-Making

Analyze trends and predict outcomes.
Handle complex scenarios.
Provide actionable insights

Scalability

Scale operations efficiently.
Reduce operational costs.
Support business growth.

24/7 Availability

Operate continuously.
Ensure prompt task completion.
Provide round-the-clock service

Use case examples

Personal productivity

Research

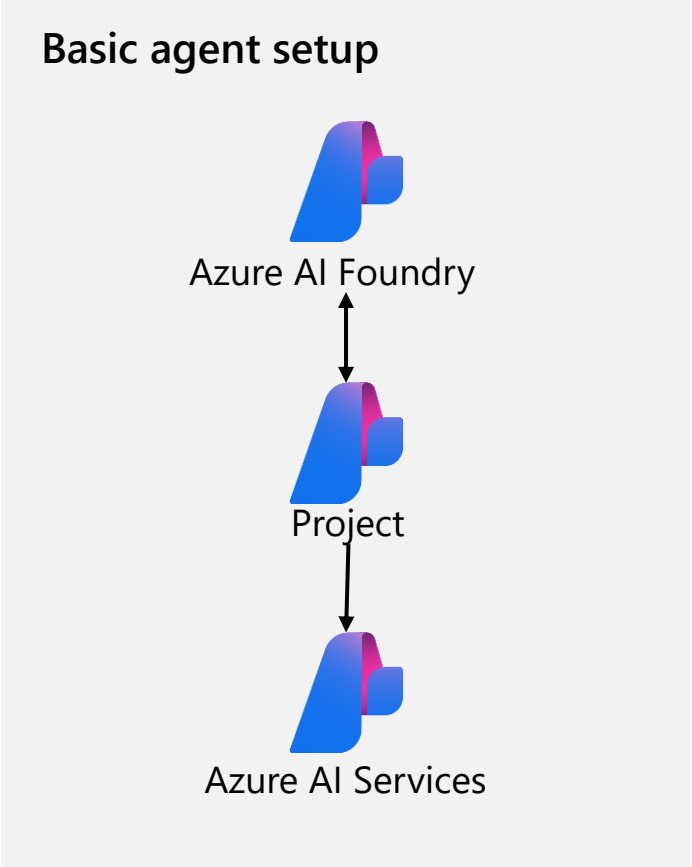
Sales

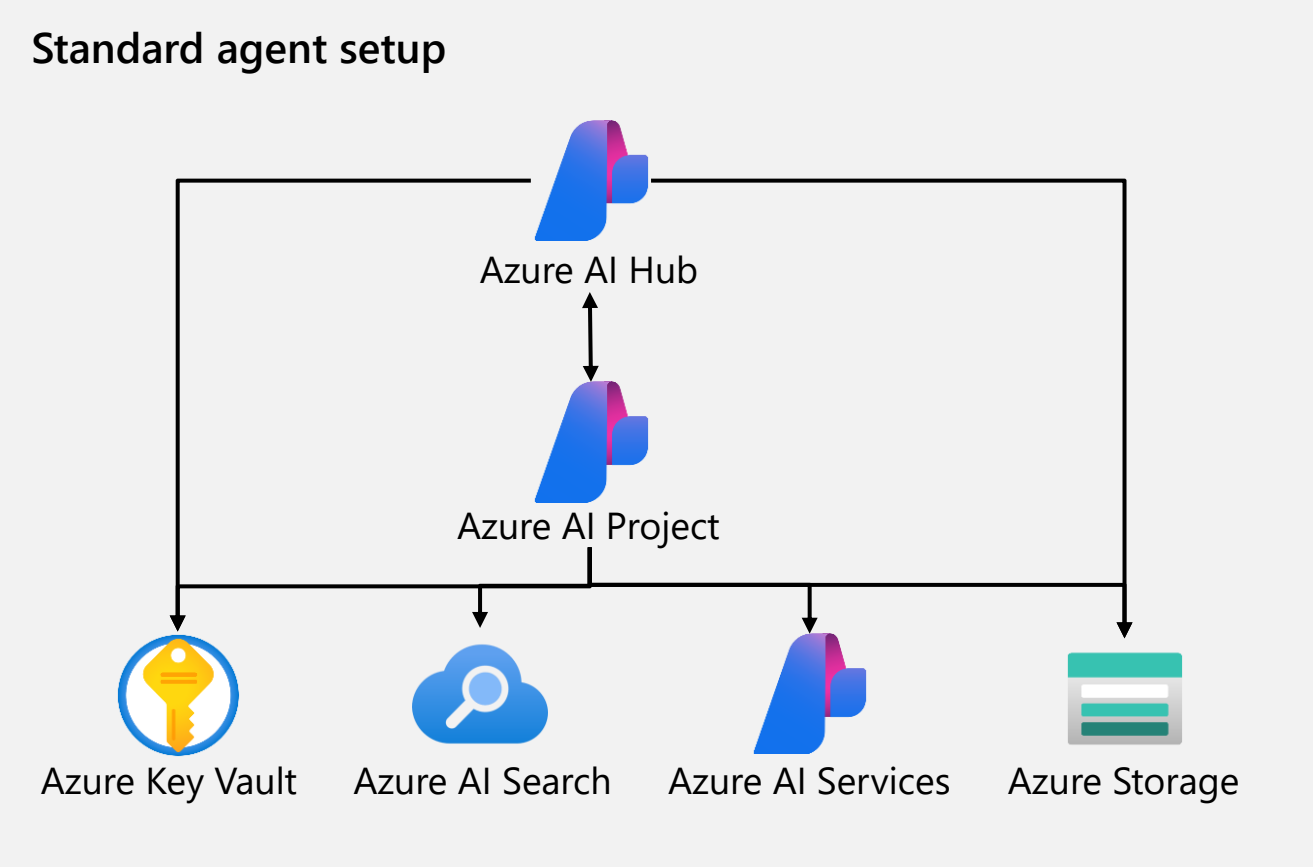
Customer service

Software development

Agent resource setup

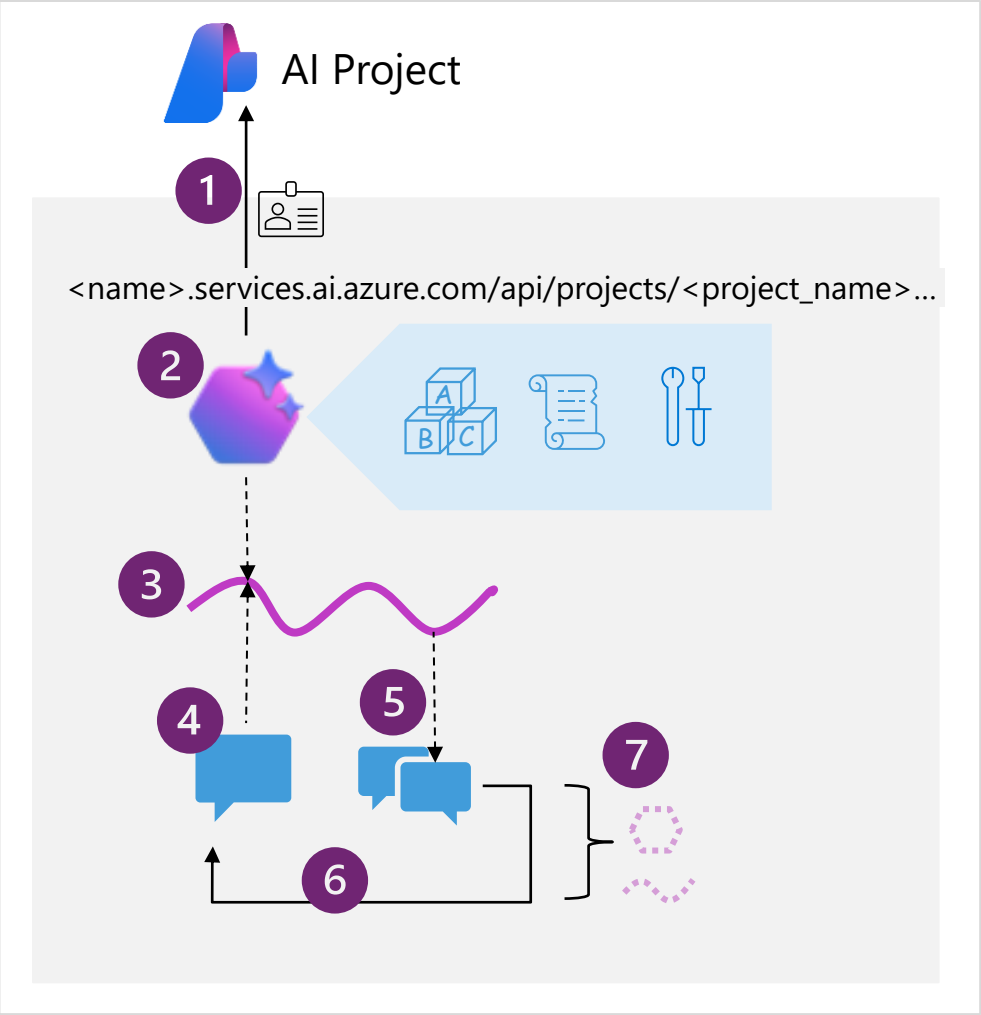
- Use bicep templates for basic and standard setup
- Or create resources for a basic setup in Azure AI Foundry portal





Using the SDK to chat with an agent

1. Connect to the AI Foundry *project*
2. Get an existing *agent*, or create one specifying:
   - *Model*
   - *Instructions*
   - *Tools* and *resources* it can use
3. Create a *thread* for the chat session
4. Add *messages* to the thread and invoke it with the agent
5. Check the thread *status*, and when ready, retrieve the messages. The thread includes all user and agent messages plus any other data that was generated (e.g. files created by tools)
6. Repeat…
7. When finished, delete the agent and the thread



Using tools with your agent

Knowledge tools

- **Bing Search**: Uses Bing search results to ground prompts with real-time live data from the web
- **File search**: Grounds prompts with data from files in a vector store
- **Azure AI Search**: Grounds prompts with data from Azure AI Search query results
- **Microsoft Fabric**: Uses the Fabric Data Agent to ground prompts with data from your Fabric data stores
- **Licensed data**: Integrate third-party licensed data by using the OpenAPI Spec action tool

Action tools

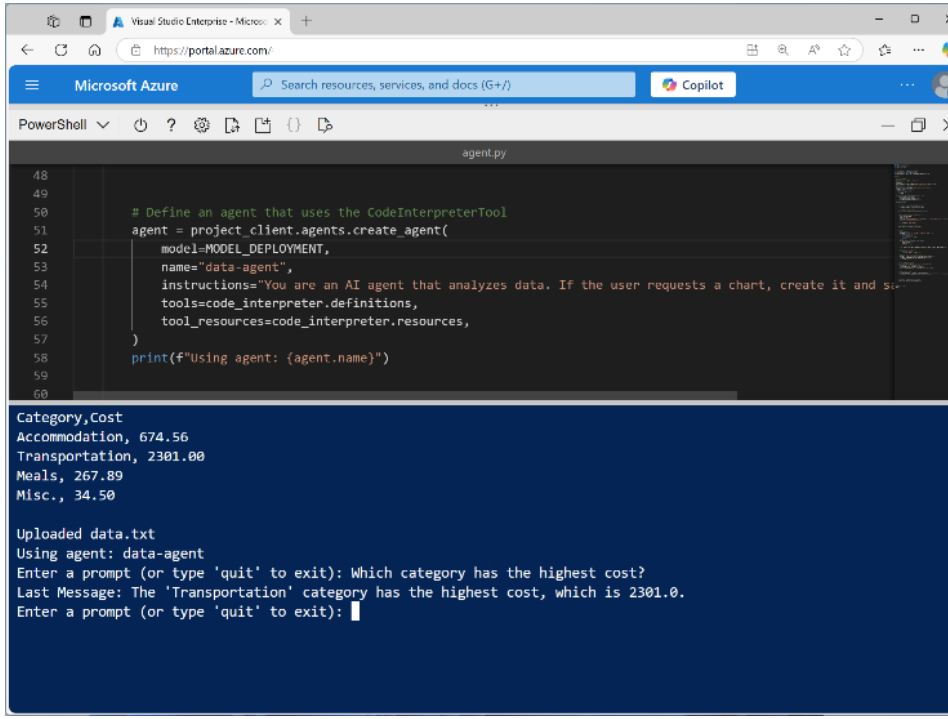
- **Code Interpreter**: Sandbox for model-generated Python code that can access and process uploaded files
- **Function**: Calls your custom function code – you must provide function definitions and implementations
- **Azure Function**: Calls code in serverless Azure Functions
- **OpenAPI Spec**: Calls external APIs based on the OpenAPI 3.0 spec

Exercise

Develop an AI agent

In this exercise, you'll use the Foundry Agent SDK to:

- Create an agent that uses a code completion tool
- Invoke a chat session with the agent
- Retrieve chat messages and output files



Knowledge check

1 **What is the first step in setting up Azure AI Foundry Agent Service?**

- ☐ Deploy a compatible model
- ☑ Create a project
- ☐ Make API calls using SDKs

2 **Which element of an agent definition is used to specify its behavior and restrictions?**

- ☐ Model
- ☑ Instructions
- ☐ Tools

3 **Which tool should you use to enable an agent to dynamically generate code to perform tasks or access data in files?**

- ☑ Code Interpreter
- ☐ Azure Functions
- ☐ Azure AI Search

Integrate custom tools into your agent

Why use custom tools?

Enhance productivity

Automate repetitive tasks and streamline workflows

Improve accuracy

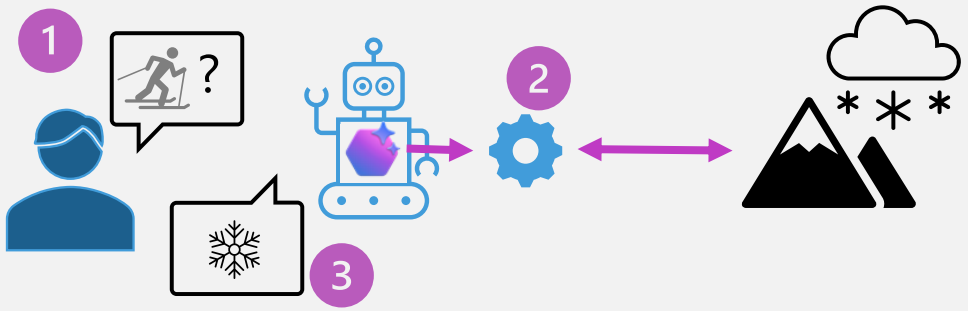
Provide precise and consistent outputs with explicit code

Create tailored solutions

Address specific business needs and optimize processes

Example

1. A user asks an agent about the weather conditions in a ski resort
2. The agent determines that is has access to a tool that can use an API to get meteorological information, and calls it
3. The tool returns the weather report, and the agent informs the user



Options for integrating custom tool functionality

Function calling

- Provide a set of callable functions in your agent code (multiple languages supported)
- The agent dynamically identifies appropriate functions based on their definitions
- Useful for integrating custom logic and workflows into AI agents

Azure Functions

- Provide trigger and binding details for Azure functions you want the agent to call
- Useful for integrating scalable, event-driven serverless application logic into an agent solution

OpenAPI Spec

- Connect to an external function using the OpenAPI 3.0 standard
- Useful for integrating custom or third-party services into an agent solution

Knowledge check

1 **What are custom tools, and how can they help you develop effective agents with Foundry Agent Service?**

- ☑ Callable functions that an agent can use to extend its capabilities
- ☐ Extensions for Visual Studio Code that make it easier to create and deploy agents
- ☐ Fine-tuned models that the agent can use to generate custom output

2 **You need to integrate functionality from an OpenAPI 3.0-based web service into an agent solution. What should you do?**

- ☐ Add the JSON schema of the web service to the agent's instructions
- ☐ Rewrite the web service as a Python function and hard-code it in your agent app
- ☑ Add the web service as an OpenAPI specification tool to the agent definition

3 **Your agent application code includes a local function that you want the agent to call. What kind of tool should you add to the agent's definition?**

- ☑ Function calling
- ☐ Code interpreter
- ☐ Azure Functions

Thank You